Ch. 11 Notes

B-trees

Why B-trees

Binary BSTs are memory-bound on pointer chases – each hop is a cache miss. A B-tree stuffs many keys per node so each disk / cache block read pays off: height becomes $O(\log_m n)$ for order m. The standard file-system / database index.

Definition (order m)

For every non-root internal node:

- Has k keys, $\lceil m/2 \rceil - 1 \leq k \leq m - 1$.
- Has $k + 1$ children.
- Keys within a node are sorted; child i holds everything between key $i - 1$ and key i.
- All leaves at the *same depth*.

Root relaxed: may have as few as 1 key. **2-3-4 tree** = order 4 (keys 1–3, children 2–4).

Node skeleton

```
struct BTNode {
    bool isLeaf;
    std::vector<int>        keys;      //
         up to m-1
    std::vector<BTNode*>    children;  //
         up to m
};
```

Node arrays kept sorted; binary-search within a node.

Height

Fully packed: $n \approx m^{h+1} \Rightarrow h = O(\log_m n)$. For $m = 100$ and $n = 10^9$: $h \approx 5$.

Search

Within a node, binary-search for the key or the child slot.

```
int i = 0;
while (i < node->keys.size() && key >
    node->keys[i]) ++i;
if (i < node->keys.size() && key == node
    ->keys[i]) return hit;
if (node->isLeaf) return miss;
return search(node->children[i], key);
```

$O(m \log_m n) = O(\log n)$, but every "step" is one block read.

Split (the core insert primitive)

Node with $m - 1$ keys (full) needs to grow:

1. Take the *median* key; push it up into the parent.
2. Left sibling gets the left half; new right sibling gets the right half. Children split accordingly.
3. If parent is now full, split it too – cascades up.

Root split creates a new root $\rightarrow$ tree grows by one level (the only way B-trees grow).

Insert (preemptive split)

Descend toward the leaf; whenever you're about to enter a full node, split it first. Then insert into the leaf – guaranteed to have room.

- One downward pass; no retrace.
- $O(\log n)$ node reads, $O(1)$ splits per level on average.

Rotation (sibling borrow)

Transfers one key to an under-full node from a sibling through the parent, preserving sort order.

- **Right-rotate on node**: borrow from left sibling via parent.
- **Left-rotate on node**: borrow from right sibling.

Both are $O(m)$.

Fusion (merge) – the remove primitive

Under-full node + under-full sibling $\Rightarrow$ pull the separator key down from the parent and concatenate the two into one node. If the parent is now under-full, recurse upward. Root fusion collapses a level – the only way B-trees shrink.

Remove (preemptive merge)

Descending to the key:

- Before entering an under-full child, **rotate** from a sibling if possible, else **fuse** with a sibling.

At the leaf, simply delete the key. Internal-node remove: replace with in-order predecessor / successor from the appropriate subtree, then remove that leaf key.

Insert vs. remove primitives

direction	primitive
grow, make room	split (node $\rightarrow$ two + median up)
shrink, share	rotate (neighbor lends one)
shrink, collapse	fusion (merge with neighbor)

Cost

All three operations: $O(\log_m n)$ node reads; within a node $O(\log m)$ search or $O(m)$ insert shift. If m matches the disk block, height is tiny ($\leq$ 5–6 for billions of keys).

B-tree vs. B+ tree

B+ keeps all data in leaves; internal nodes are pure routing (just keys). Leaves form a linked list. Range queries are then a single traversal along the leaf list. Every production database (MySQL InnoDB, PostgreSQL, SQLite, LevelDB) ships a B+.

Top gotchas

- Using the wrong *order* definition – "order m" means m children max, $m - 1$ keys max. Stay consistent.
- Forgetting the root is the only node allowed to have $< \lceil m/2 \rceil - 1$ keys.
- Splitting post-insertion instead of preemptively – doable but forces a second pass.
- Remove without rotating/fusing first – leaves a node with too few keys; violates the invariant.
- Using B-tree where a hash index would do. B-trees earn their keep on range queries and sorted iteration.